

הרצאה 11 – Node.js & Client

בל הזכויות שמורות לקרן יעקב 2025

תוכן עניינים

1. Express - המשך וחיבור צד הלקוח
2. git ignore
3. משתני סביבה
4. Logger
5. Database - התחברות ושימוש
6. Express Router



1. Express – המשך

וחיבור צד הלקוח

app.use



```
app.use(express.json());  
app.use(express.urlencoded({extended: true}));
```

- מתודה של אקספרס שמגדירה לנו middlewares.
- דוגמה 1 - (ראינו בהרצאה הקודמת) למה משמש הקוד?
באילו מצבים הם יופעלו?

- דוגמה 2 - אפשר להגדיר middleware שיופעל רק בנתיבים מסויימים

```
19 app.use('/posts/:postId', (req, res, next) => {  
20   console.log(`Request Type: ${req.method} and postId: ${req.params.postId}`);  
21   next();  
22 });
```

app.use - המשך

- מתודה של אקספרס שמגדירה לנו middlewares.

```
app.use(express.json());  
app.use(express.urlencoded({extended: true}));
```

- דוגמה 1 - (ראינו בהרצאה הקודמת) למה משמש הקוד?
באילו מצבים הם יופעלו?

Built-in middleware

- `express.json` parses incoming requests with JSON payloads. **NOTE: Available with Express 4.16.0+**
- `express.urlencoded` parses incoming requests with URL-encoded payloads. **NOTE: Available with Express 4.16.0+**

- דוגמה 2 - אפשר להגדיר middleware שיופעל רק בנתיבים מסויימים

```
19 app.use('/posts/:postId', (req, res, next) => {  
20   console.log(`Request Type: ${req.method} and postId: ${req.params.postId}`);  
21   next();  
22 });
```

GET



localhost:8081/posts/2

```
[nodemon] starting `node run index.js`  
listening on port 8081  
Request Type: GET and postId: 2
```

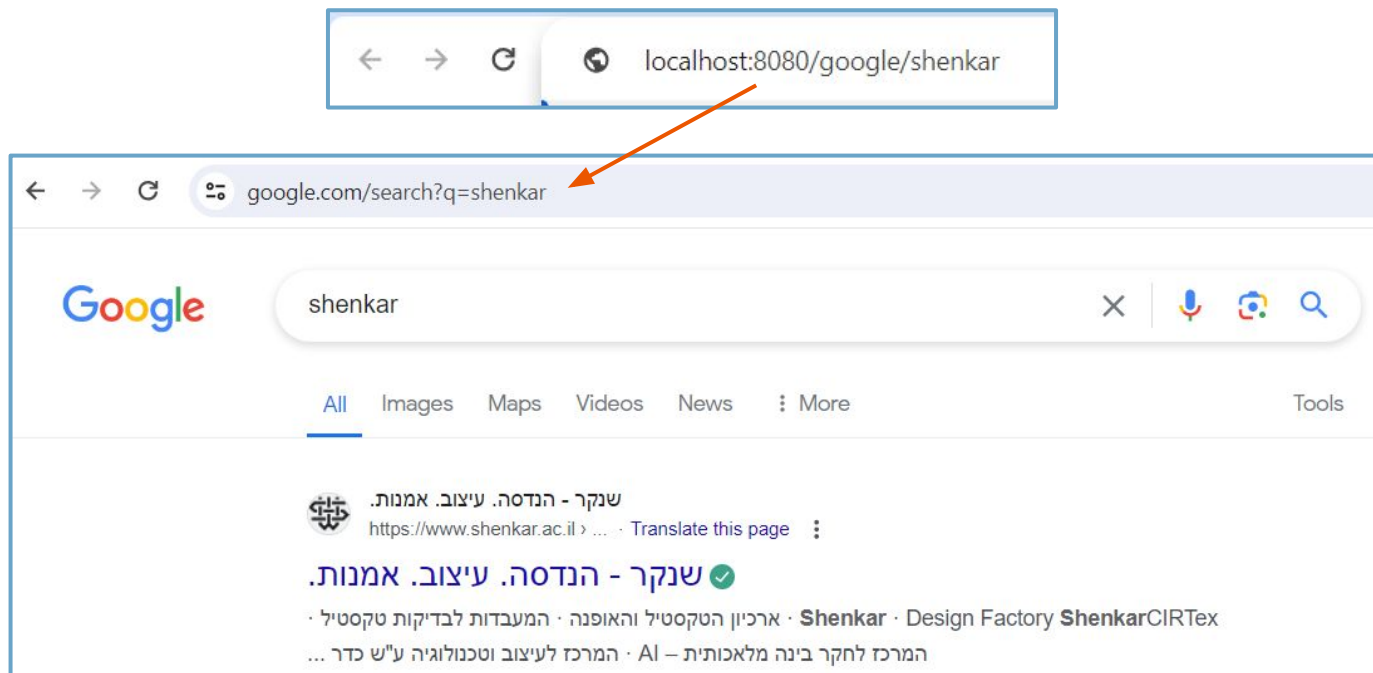
res.redirect

- מתודה של אובייקט ה-response של אקספרס והיא תפנה את המשתמש לכתובת המצויינת בפרמטר הראשון.
- איך צד הלקוח פונה לשרת? מה השרת מחזיר?

```
index.js X
Shenkar2024Keren > 10.1-Nodejs-lec > redirect > index.js > ...
1  const express = require('express');
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.get("/google/:searchQuery", (req, res) => {
6      |   res.status(301).redirect(`https://www.google.com/search?q=${req.params.searchQuery}`);
7      |   }
8  });
9
10 app.listen(port);
11 console.log(`listening on port ${port}`);
```

- שימו לב, שינוי מיקום של דפים **לצמיתות** מחייב החזרת סטטוס 301

res.redirect - המשך



res.set

- מתודה של אובייקט ה-response של אקספרס והיא מאפשרת להגדיר את ה-HTTP Header של התשובה ללקוח

```
// Setting the response
res.set({
  'Content-Type': 'application/json'
});
```

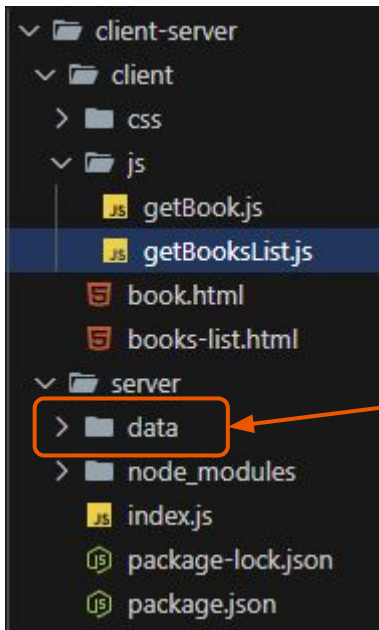
- נראה עוד דוגמה בהמשך

חיבור צד הלקוח לשרת

- כמו שלמדנו, כשהדפדפן פונה לשרת ומקבל תשובה - הדפדפן הוא צד הלקוח
- כשאנחנו משתמשים ב-Postman כדי לעשות קריאות לשרת - הפוסטמן הוא צד הלקוח
- את אותן קריאות אפשר לעשות מתוך הפרויקט שלנו, קובץ JS שיבצע קריאות AJAX

- נשתמש בתרגיל ספרים שתיכנתם בתרגול ונשנה אותו, כך שה-json ישב בצד השרת ולא בצד הלקוח.
- תזכורת - ה-json מדמה לנו דטבייס לא רלציוני.

מצד שמאל אנחנו רואים את מבנה הפרויקט החדש - שתי תיקיות: אחת לשרת ואחת לצד הלקוח. תיקיית data ובה ה-jsons עברו לצד השרת, כך שאם נפעיל עכשיו המידע לא יטען



חיבור צד הלקוח לשרת - הבאת מידע מהשרת

```
19 window.onload = () => {          צד לקוח
20     fetch("http://127.0.0.1:8080/books")
21     .then(response => response.json())
22     .then(data => showData(data));
23 }
```

```
JS getBooksList.js  JS index.js  X
Shenkar2024Keren > 10.1-Nodejs-lec > client-server > server > JS index.js > ...
1  const express = require('express');
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.get("/books", (req, res) => {
6      console.log("Got the request from the client side");
7      res.json({"response": "ok"});
8  });
9
10 app.listen(port);
11 console.log(`listening on port ${port}`);
```

צד שרת

- נרים את השרת ונשנה את פונקציית ה-fetch כדי שתביא לנו את המידע מצד השרת
- נכתוב קוד בשרת שיקבל את הבקשה הזו. נרשום שם באופן זמני הדפסה לקונסול ותשובה לצד הלקוח
- נבצע את הבקשה...

```
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
listening on port 8080
Got the request from the client side
```

Console

2 Issues: 1 1

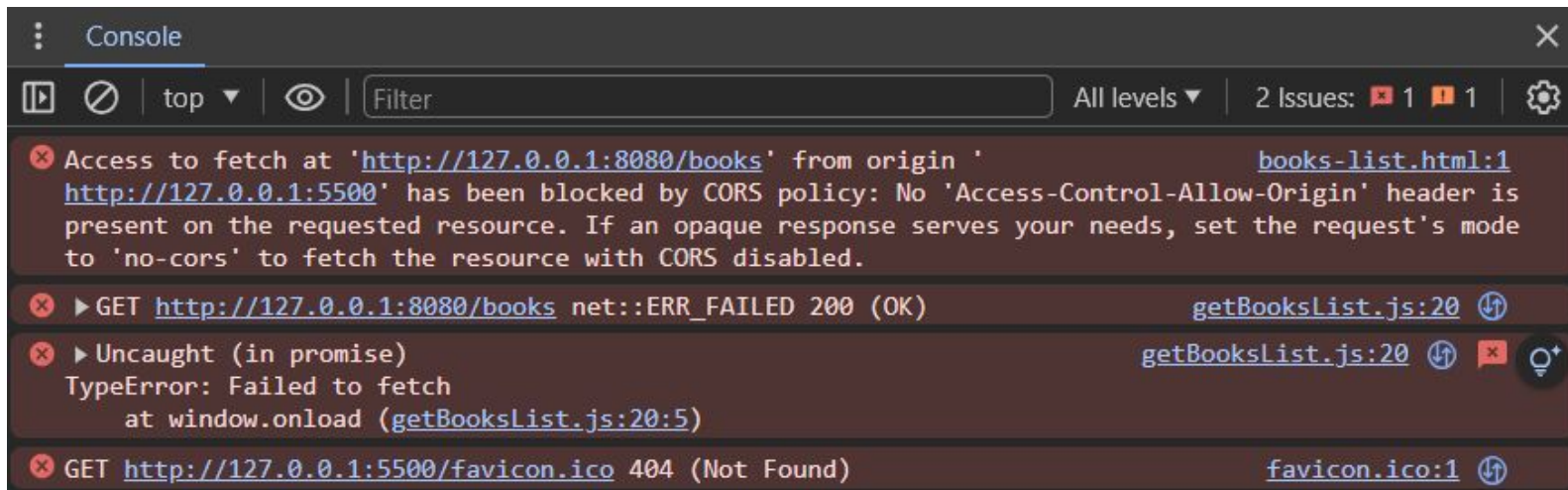
- ✖ Access to fetch at 'http://127.0.0.1:8080/books' from origin 'http://127.0.0.1:5500' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. If an opaque response serves your needs, set the request's mode to 'no-cors' to fetch the resource with CORS disabled. [books-list.html:1](#)
- ✖ ▶ GET http://127.0.0.1:8080/books net::ERR_FAILED 200 (OK) [getBooksList.js:20](#)
- ✖ ▶ Uncaught (in promise) TypeError: Failed to fetch at window.onload (getBooksList.js:20:5) [getBooksList.js:20](#)
- ✖ GET http://127.0.0.1:5500/favicon.ico 404 (Not Found) [favicon.ico:1](#)

חיבור צד הלקוח לשרת - שגיאת CORS

- CORS = Cross Origin Resource Sharing
- כדי לאפשר בקשות AJAX בין שני דומיינים שונים, עלינו להגדיר זאת במפורש, מסיבות אבטחה.
- איך נאפשר? בצד השרת נחזיר כמה הדרים כמו

Access-Control-Allow-Origin

שבו נציין לאילו דומיינים מותרת הגישה או * לכולם.



```
⋮ Console
[Icons] top [Filter] All levels 2 Issues: 1 1 [Settings]

✖ Access to fetch at 'http://127.0.0.1:8080/books' from origin 'http://127.0.0.1:5500' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource. If an opaque response serves your needs, set the request's mode to 'no-cors' to fetch the resource with CORS disabled. books-list.html:1

✖ ▶ GET http://127.0.0.1:8080/books net::ERR_FAILED 200 (OK) getBooksList.js:20

✖ ▶ Uncaught (in promise)
TypeError: Failed to fetch
    at window.onload (getBooksList.js:20:5)

✖ GET http://127.0.0.1:5500/favicon.ico 404 (Not Found) favicon.ico:1
```

חיבור צד הלקוח לשרת - פתרון CORS

- מה הקוד הבא עושה? איפה נוסify אותו?

```
6 app.use((req, res, next) => {  
7   res.set({  
8     'Access-Control-Allow-Origin': '*',  
9     'Access-Control-Allow-Headers': 'Origin, X-Requested-With, Content-Type, Accept',  
10    'Access-Control-Allow-Methods': "GET, POST, PUT, DELETE",  
11    'Content-Type': 'application/json'  
12  });  
13  next();  
14 });
```

- נריץ מחדש, כבר אין שגיאת CORS. עכשיו נחזור לטפל בקריאת ה-JSON בצד השרת
- אפשר להשתמש במודול cors מ-npm, לא נלמד במסגרת הקורס

חיבור צד הלקוח לשרת - טעינת ה-JSON

```
4 const booksData = require('./data/books.json');
5
6 app.use((req, res, next) => {
7   res.set({
8     'Access-Control-Allow-Origin': '*',
9     'Access-Control-Allow-Headers': 'Origin, X-Requested-With, Content-Type, Accept',
10    'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE',
11    'Content-Type': 'application/json'
12  });
13  next();
14 });
15
16 app.get("/books", (req, res) => {
17   res.json(booksData);
18 });
```

Books

Products

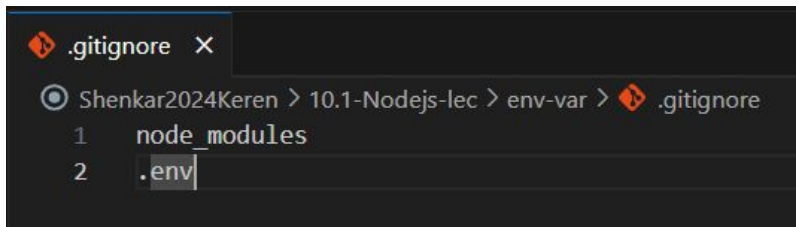
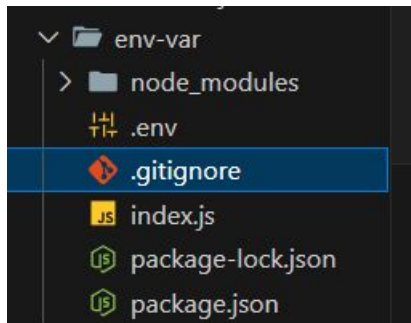
- [Cinderella](#)
- [Snow White](#)
- [Mickey Mouse](#)



git ignore .2

git ignore

- נועד כדי לאפשר קבצים בתיקיית הפרוייקט שלא נעלה לשרת הגיטהאב
- לדוגמה: תיקיית node_modules או קובץ .env של משתני הסביבה (לעיתים יש שם סיסמאות)
- בתיקייה הראשית של הפרוייקט ניצור קובץ ללא שם, עם סיומת gitignore ובו נשים את הקבצים ו/או התיקיות שלא נרצה להעלות לגיטהאב
- עוד מידע - <https://git-scm.com/docs/gitignore>





3. משתני סביבה

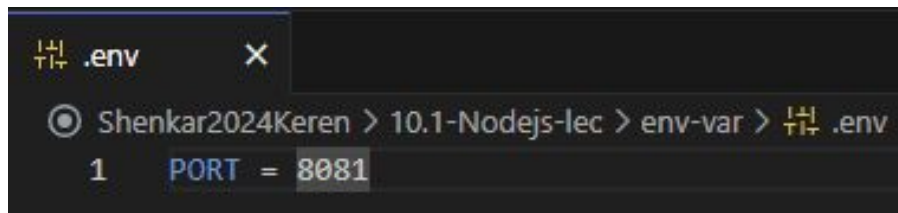
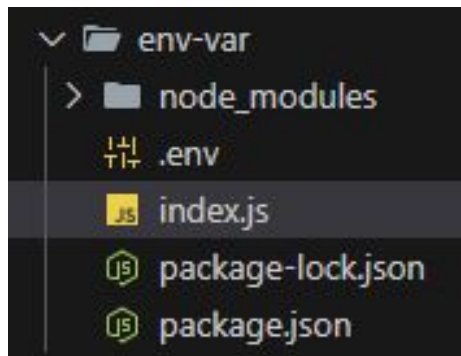
משתני סביבה – Environment variables

- משתני סביבה: משתנים גלובליים בסביבה (שרת) של הקוד
- בעיה: בשרת המקומי שלי אני רוצה להאזין ב- port 8080 אבל השרת בענן מאזין ב- port 3000
- פתרון: נגדיר שה- port תלוי בסביבה (=בשרת)
- נשתמש במשתני סביבה:

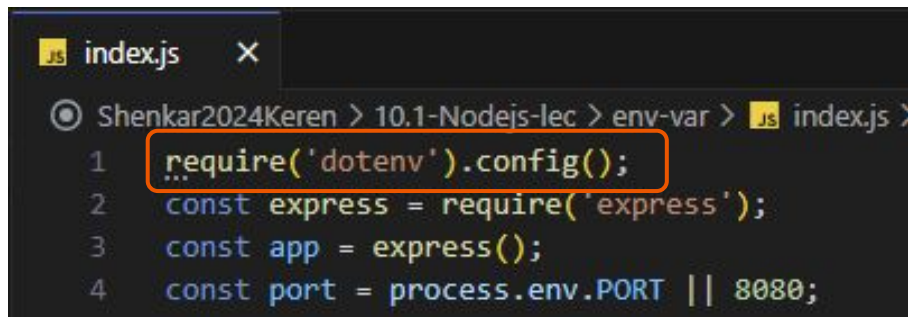
```
JS index.js x
JS index.js > ...
1  const express = require('express');
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.listen(port);
6
7  console.log(`listening on port ${port}`);
```

משתני סביבה - מימוש

- נוריד את המודול dotenv מתוך npm
- בתיקייה הראשית של הפרוייקט ניצור קובץ ללא שם, עם סיומת env, ונשים בו את המשתנים התלויים בסביבה. אם יש מספר זוגות, כל זוג יהיה בשורה משלו ללא פסיקים ביניהם.



- נוסיף את שורת האיתחול הכי מוקדם שאפשר בקוד:



משתני סביבה - מימוש - המשך

```
.env x
Shenkar2024Keren > 10.1-Nodejs-1ec > env-var > .env
1 PORT = 8081
```

```
index.js x
Shenkar2024Keren > 10.1-Nodejs-1ec > env-var > index.js >
1 require('dotenv').config();
2 const express = require('express');
3 const app = express();
4 const port = process.env.PORT || 8080;
```

web 2024\Shenkar2024Keren\10.1-Nodejs-1ec\env-var> npm run dev

```
> env-var@1.0.0 dev
> node index.js

listening on port 8081
█
```



Logger .4

Logger – Morgan

morgan 

1.10.0 • Public • Published 4 years ago

 Readme

 Code Beta

 5 Dependencies

 9,194 Dependents

 26 Versions

morgan

npm v1.10.0 downloads 19.7M/month travis 404 coverage 100%

HTTP request logger middleware for node.js

Named after **Dexter**, a show you should not watch until completion.

API

```
var morgan = require('morgan')
```

morgan(format, options)

Create a new morgan logger middleware function using the given `format` and `options`. The `format` argument may be a string of a predefined name (see below for the names), a string of a format string, or a function that will produce a log entry.

The `format` function will be called with three arguments `tokens`, `req`, and `res`, where `tokens` is an object with all defined tokens, `req` is the HTTP request and `res` is the HTTP

Install

```
> npm i morgan
```

Repository

 github.com/expressjs/morgan

Homepage

 github.com/expressjs/morgan#readme

Weekly Downloads

4,600,540

Version

1.10.0

License

MIT

Unpacked Size

29.7 kB

Total Files

5

Issues

17

Pull Requests

9

Last publish

4 years ago

<https://www.npmjs.com/package/morgan>

דוגמה יש בעמוד ה-npm של הספרייה.

לפי הדוגמה שבדוקומנטציה, הטמיעו בפרוייקט



– Database .5

התחברות ושימוש

SQL



- SQL = Structured Query Language
- נשתמש ב-SQL כדי לגשת ולנהל את המידע בדטאבייס

- במסגרת הקורס נשתמש ב-4 פעולות CRUD שלהן יש הצהרות מקבילות ב-SQL:

Create – `INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);`

Read – `SELECT column1, column2, ...
FROM table_name;`

Update – `UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;`

Delete – `DELETE FROM table_name WHERE condition;`

async / await

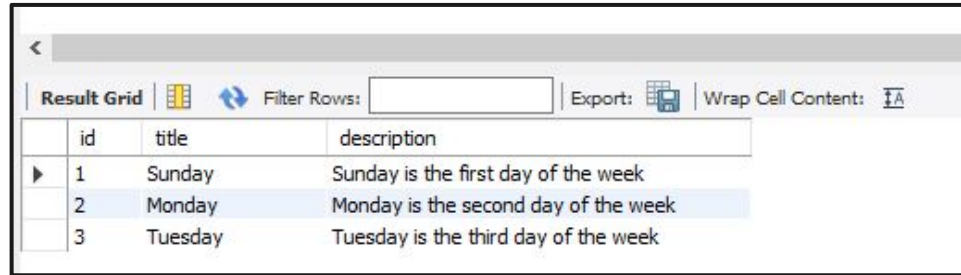
- כתיבה אחרת ל-Promise, בצורה יותר ברורה ונקייה.
- עלינו להצהיר על הפונקציה כאסינכרונית ע"י async וכך נוכל בפונקציה עצמה להשתמש ב-await כדי לחכות לתשובה כשתגיע (ניתן לא להגדיר await או להגדיר כמה בפונקציה).
- פעולות מול הדטבייס יכולות לקחת זמן ואם לא נשתמש ב-await async לא תמיד התשובה תגיע לפני שנחזיר אותה לצד הלקוח.

● דוגמה:

```
1 async function showAvatar() {  
2  
3   // read our JSON  
4   let response = await fetch('/article/promise-chaining/user.json');  
5   let user = await response.json();
```


דטבייס

- את ההכנה לשיעור סיימתם כשיצרתם טבלה עם 3 רשומות:



The screenshot shows a web interface with a table titled 'Result Grid'. The table has three columns: 'id', 'title', and 'description'. It contains three rows of data. The second row, representing 'Monday', is highlighted in blue. Above the table, there is a 'Filter Rows' input field and buttons for 'Export' and 'Wrap Cell Content'.

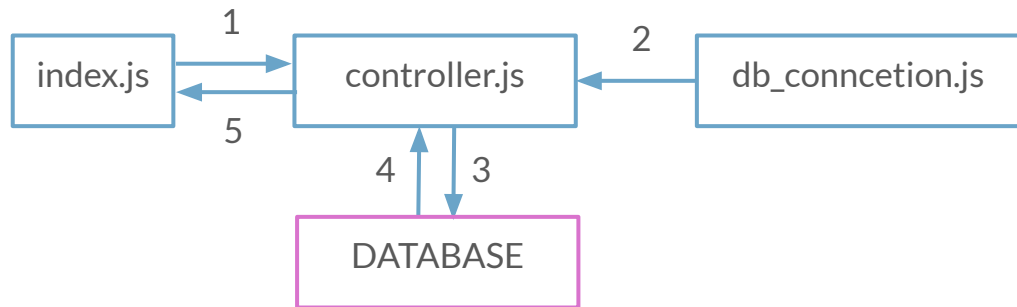
	id	title	description
▶	1	Sunday	Sunday is the first day of the week
	2	Monday	Monday is the second day of the week
	3	Tuesday	Tuesday is the third day of the week

- נשתמש בחיבור לדטבייס ובטבלה שיצרנו כדי לבצע פעולות CRUD על הדטבייס

חשוב!

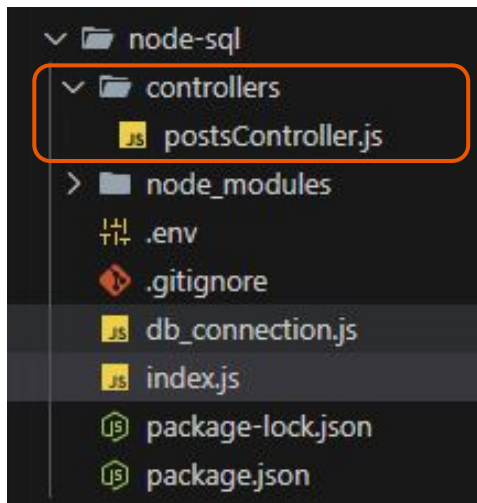
- המימוש של צד השרת שנראה עכשיו הוא זמני! בשביל ההבנה נעשה את המימוש בשלבים.
- בהמשך נשתמש גם ב-Router, ננהל שגיאות, נתעד לוגים...

- סכמה כללית של זרימת המידע בצד השרת (זמני):



חיבור לדטבייס 1 - יצירת קובץ קונטרולר

- את הלוגיקה לא נכניס לקובץ הראשי של השרת ולכן ניצור תיקיית controllers בתיקיית הפרוייקט ושם ניצור את postsController.js



- הקריאה לקונטרולר תעשה מתוך קובץ index.js

```
6  const { postsController } = require('./controllers/postsController.js');
```

- והשימוש יעשה (כרגע) מתוך הראוטים השונים, לדוגמה:

```
19  app.get("/posts", async (req, res) => {  
20    res.status(200).send(await postsController.getPosts());  
21  });
```



חיבור לדטבייס 1 - יצירת קובץ קונטרולר - המשך

Shenkar2024Keren > 10.1-Nodejs-lec > node-sql > controllers > postsController.js > ...

```
1  exports.postsController = {
2    // GET localhost:8081/posts
3    async getPosts() {
4      const { dbConnection } = require('../db_connection');
5      const connection = await dbConnection.createConnection();
6
7      const [rows] = await connection.execute(`SELECT * from tbl_99_posts`);
8
9      connection.end();
10
11     return rows;
12   },
13   // GET localhost:8081/posts/2
14   async getPost(postId) { ...
23   },
24   // POST localhost:8081/posts
25   async addPost(body) { ...
34   },
35   // PUT localhost:8081/posts/5
36   async updatePost(body, postId) { ...
45   },
46   // DELETE localhost:8081/posts/5
47   async deletePost(postId) { ...
56   }
57 };
```

● נתמקד בפלואו של הבאת כל הפוסטים (GET)

חיבור לדטבייס 1 - יצירת קובץ החיבור לדטבייס

```
Shenkar2024Keren > 10.1-Nodejs-lec > node-sql >  db_connection.js > ...  
1  exports.dbConnection = {  
2    async createConnection() {  
3      const mysql = require('mysql2/promise');  
4  
5      const connection = await mysql.createConnection({  
6        host      : process.env.DB_HOST,  
7        user      : process.env.DB_USERNAME,  
8        password  : process.env.DB_PASSWORD,  
9        database  : process.env.DB_NAME  
10     });  
11  
12     return connection;  
13   }  
14 }
```

- החיבור לדטבייס משותף לכולם.
- בכל פעם שנרצה לגשת לדטבייס, נפתח את החיבור ובסוף השימוש נסגור אותו.
- מאיפה מגיעים פרטי החיבור לדטבייס?

חיבור לדטבייס 1 - קובץ package.json

```
Shenkar2024Keren > 10.1-Nodejs-lec > node-sql > package.json > ...
1  {
2    "name": "sql",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1",
7      "dev": "nodemon run index.js"
8    },
9    "author": "Keren",
10   "license": "ISC",
11   "description": "",
12   "dependencies": {
13     "dotenv": "^16.4.5",
14     "express": "^4.19.2",
15     "mysql2": "^3.10.1"
16   }
17 }
```

- מה עלינו להוסיף לפרוייקט בשביל החיבור לדטבייס?
- מה כל dependency עושה?

דוקומנטציה של mysql2/promise
<https://www.npmjs.com/package/mysql2-promise>

חיבור לדטבייס 1 - הרצה

GET localhost:8081/posts

localhost:8081/posts

GET localhost:8081/posts

Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Query Params

KEY	VALUE	DESCRIPTION

Body Cookies Headers (10) Test Results

Status: 200 OK Time: 1448 ms Size: 724 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   {
3     "id": 1,
4     "title": "Sunday",
5     "description": "Sunday is the first day of the week"
6   },
7   {
8     "id": 2,
9     "title": "Monday",
10    "description": "Monday is the second day of the week"
11  },
12  {
13    "id": 3,
14    "title": "Tuesday",
15    "description": "Tuesday is the third day of the week"
16  },
17  {
18    "id": 4,
19    "title": "This is a new title 2",
20    "description": "This is a new description 2"
```

נרים את השרת
נבצע קריאה מתוך Postman



	id	title	description
▶	1	Sunday	Sunday is the first day of the week
	2	Monday	Monday is the second day of the week
	3	Tuesday	Tuesday is the third day of the week
	4	This is a new title 2	This is a new description 2



Express Router .6



Express Router

- ה-Express Router הוא קלאס שעוזר לנו ליצור ולנהל את הראוטים
- הרעיון: כל בקשת http מגיעה ל-router והוא מפנה את הבקשה לקונטרולר המתאים
- ככל שהמערכת גדלה נצטרך לנהל יותר ראוטים, יותר מודלים... נצטרך עוד קונטרולרים ועוד ראוטרים

```
const express = require("express");
const router = express.Router();

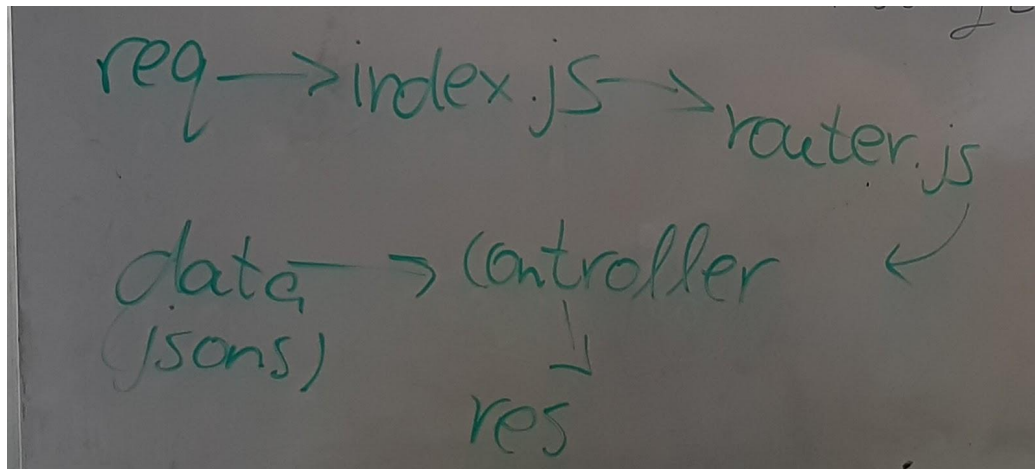
router.get("/", (req, res) => {
  // CODE GOES HERE
});
router.get("/:id", (req, res) => {
  // CODE GOES HERE
});
router.post("/", (req, res) => {
  // CODE GOES HERE
});
router.put("/", (req, res) => {
  // CODE GOES HERE
});
router.delete("/", (req, res) => {
  // CODE GOES HERE
});
module.exports = router;
```

- דוגמה לשימוש (אנחנו נייצא לקובץ נפרד, בהמשך):

Our flow

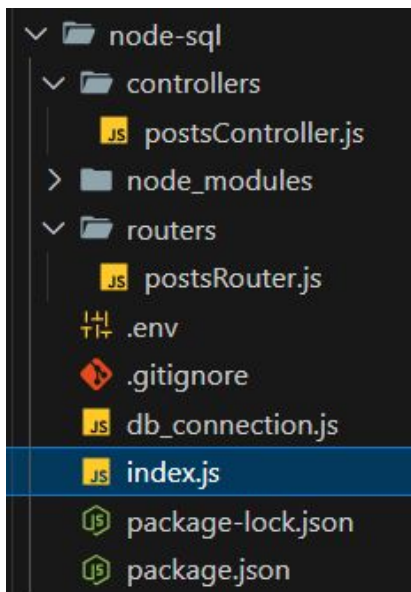


- בקשת http מגיעה ל-router מקובץ index.js
- ה-router מפנה את הבקשה לקונטרולר המתאים
- הקונטרולר עושה את הלוגיקה ומחזיר את אובייקט תשובת ה-http לצד הלקוח



הטמעת Express Router – מבנה הפרוייקט

- נשנה את הפרוייקט שלנו שיעבוד עם הראוטר
- מבחינת היררכיית התיקיות – נוספה תיקיה router ובתוכה הקובץ postsRouter.js



הטמעת Express Router – קובץ index.js

```
1 require('dotenv').config();
2 const express = require('express');
3 const app = express();
4 const port = process.env.PORT || 8080;
5
6 const { postsRouter } = require("./routes/postsRouter");
7
8 app.use(express.json());
9 app.use(express.urlencoded({extended: true}));
10
11 app.use((req, res, next) => {
12   res.set('Access-Control-Allow-Origin', '*');
13   res.set('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content-Type, Accept');
14   res.set('Access-Control-Allow-Methods', "GET, POST, PUT, DELETE");
15   res.set('Content-Type', 'application/json');
16   next();
17 });
18
19 app.use('/api/posts', postsRouter);
20
21 app.use((req, res) => {
22   res.status(400).send('Something is broken!');
23 });
24
25 app.listen(port);
26 console.log(`listening on port ${port}`);
```

● לאילו ראוטים נשלח את
הבקשות מצד הלקוח?

הטמעת Express Router – הקובץ postsRouter.js

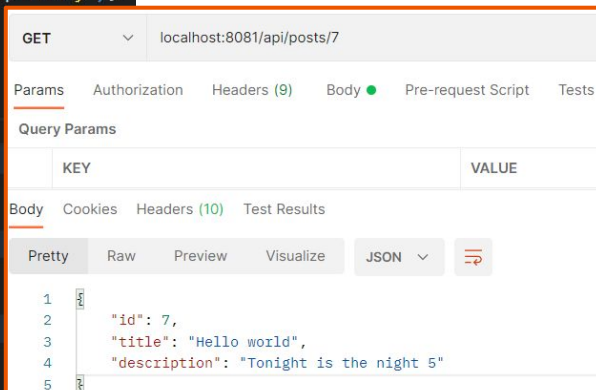
- כל אחד מהראוטים קורא למתודה אחרת של ה- postsController
- נשים לב שהאובייקטים request ו- response של express עוברים גם ונשתמש בהם בקונטרולר

```
Shenkar2024Keren > 10.1-Nodejs-lec > node-sql > routers > JS postsRouter.js > ...
1  const { Router } = require('express');
2  const { postsController } = require('../controllers/postsController.js');
3
4  const postsRouter = new Router();
5
6  postsRouter.get('/', postsController.getPosts);
7  postsRouter.get('/:postId', postsController.getPost);
8  postsRouter.post('/', postsController.addPost);
9  postsRouter.put('/:postId', postsController.updatePost);
10 postsRouter.delete('/:postId', postsController.deletePost);
11
12 module.exports = { postsRouter };
```

הטמעת Express Router – הקובץ postsController.js

- נראה את ה-get by id בדוגמה. בגלל שאפשר להשתמש ב-res, req, אנחנו יכולים לקחת מידע מהבקשה ולהחזיר את התשובה ישירות מהקונטרולר

```
1 exports.postsController = {
2   // GET localhost:8081/posts
3   > async getPosts(req, res) { ...
11 },
12   // GET localhost:8081/posts/2-
13   async getPost(req, res) {
14     const { dbConnection } = require('../db_connection');
15     const connection = await dbConnection.createConnection();
16
17     const [rows] = await connection.execute(`SELECT * from tbl_99_posts WHERE id=${req.params.postId}`);
18
19     connection.end();
20     res.json(rows[0]);
21   },
22   // POST localhost:8081/posts
23   > async addPost(req, res) { ...
32 },
33   // PUT localhost:8081/posts/5
34   > async updatePost(req, res) { ...
43 },
44   // DELETE localhost:8081/posts/5
45   > async deletePost(req, res) { ...
53 }
54 };
```



The screenshot shows a REST client interface with a GET request to `localhost:8081/api/posts/7`. The response is a JSON object with the following structure:

KEY	VALUE
id	7
title	"Hello world",
description	"Tonight is the night 5"

הוספה לדטבייס

- עד עכשיו ראינו דוגמאות לשליפה מהדטבייס
- דוגמה להוספה:

```
23  async addPost(req, res) {
24      const { dbConnection } = require('../db_connection');
25      const connection = await dbConnection.createConnection();
26
27      const { body } = req;
28      await connection.execute(`INSERT INTO tbl_99_posts (title, description) VALUES ("${body.title}", "${body.description}")`);
29
30      connection.end();
31      res.send(true);
32  },
```

עדכון הדטבייס



דוגמה לעדכון:



```
34 async updatePost(req, res) {  
35   const { dbConnection } = require('../db_connection');  
36   const connection = await dbConnection.createConnection();  
37  
38   const { body } = req;  
39   await connection.execute(`UPDATE tbl_99_posts SET title = "${body.title}", description = "${body.description}" WHERE id=${req.params.postId}`);  
40  
41   connection.end();  
42   res.send(true);  
43 },
```


מחיקה מהדטבייס

דוגמה למחיקה:

```
45  async deletePost(req, res) {  
46      const { dbConnection } = require('../db_connection');  
47      const connection = await dbConnection.createConnection();  
48  
49      await connection.execute(`DELETE FROM tbl_99_posts WHERE id=${req.params.postId}`);  
50  
51      connection.end();  
52      res.send(true);  
53  }
```